



Sistemas Distribuidos I (75.74)

Paralelización, Multicomputing y Direccionamiento

Paralelización de tareas, Multiprocessors, Multicomputing, Nombres
y Direccionamiento

Docentes

- Pablo D. Roca
- Gabriel Robles
- Franco Barreneche
- Tomás Nocetti
- Nicolás Zulaica



- **Paralelización de Tareas**
- Multi-Processors & Multi-Computing
- Nombres y Direccionamiento



Paralelización de tareas | Introducción

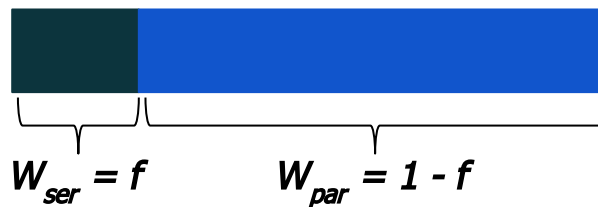
- Objetivos
 - Reducir el tiempo de cómputo de una tarea (**latencia**)
 - Incrementar la cantidad de tareas que se pueden realizar en paralelo (***throughput***)
 - Reducir la energía consumida al realizar todas las tareas
- **Camino crítico**
 - Máxima longitud de tareas secuenciales a computar
 - Define el mejor rendimiento que se puede obtener al realizar un conjunto de tareas



Paralelización de tareas | Ley de Amdahl

"...the effort expended on achieving high parallel processing rates is wasted unless it is accompanied by achievements in sequential processing rates of very nearly the same magnitude..."

- Corolario: todo trabajo de cómputo se divide en fracciones Secuenciales y Paralelas:



$$T = W_{ser} + W_{par}$$

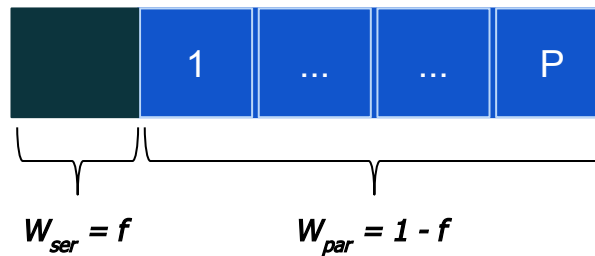


Paralelización de tareas | Ley de Amdahl

- Corolario 2: Utilizando P unidades de cómputo, el tiempo de ejecución total puede reducirse.

$$T = T_1 = W_{\text{ser}} + W_{\text{par}}$$

$$T_p = W_{\text{ser}} + W_{\text{par}} / P$$





Paralelización de tareas | Ley de Amdahl

- Se define **Speedup** como el ratio de optimización de una operación:

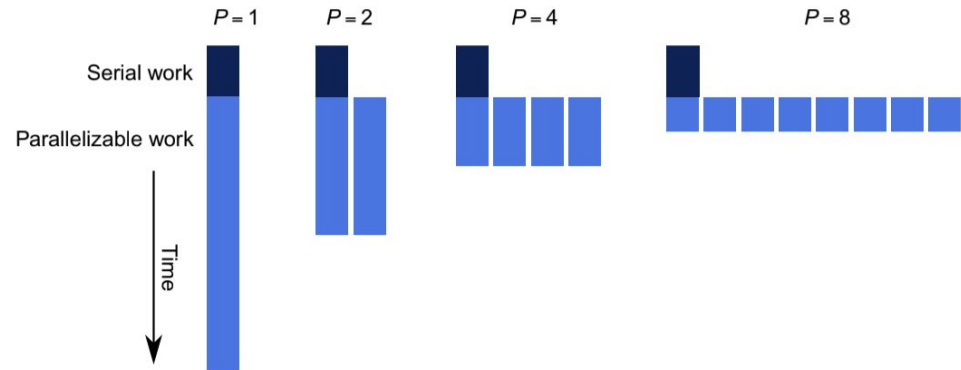
$$S_p = T_1 / T_p$$

- Reemplazando, se obtiene:

$$S_p \leq 1 / (f + (1 - f) / P)$$

- Pero con $P \rightarrow \infty$:

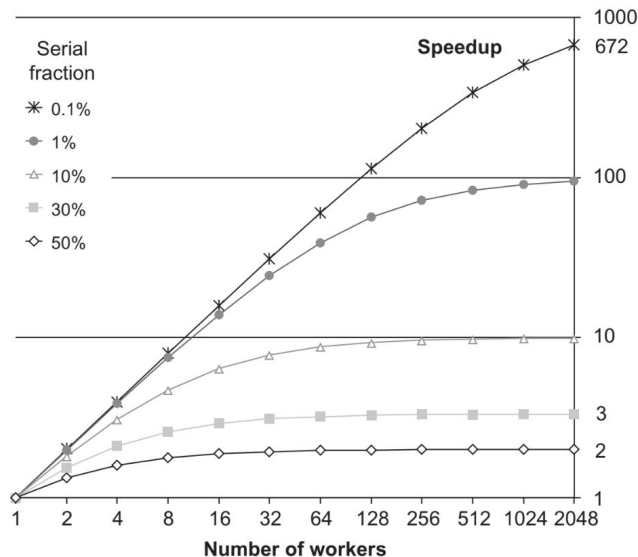
$$S_\infty \leq 1 / f$$





Paralelización de tareas | Ley de Amdahl

- *Speedup* máximo se encuentra acotado **por la fracción de tiempo que no puede ser paralelizable**
- Fracción paralelizable distribuida **uniformemente** entre procesadores



Ejemplo:

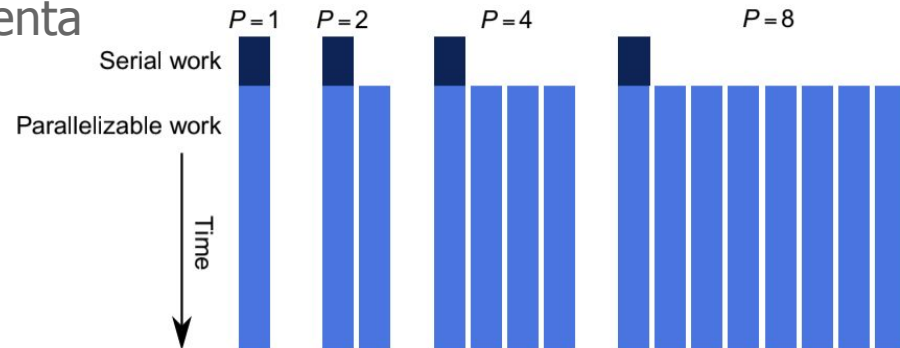
f	Processors	S_P
0.01	100	50
0.01	1000	90
0.01	10000	99
0.01	inf	100



Paralelización de tareas | Ley de Gustafson

*"Speedup should be measured by **scaling the problem** to the number of processors, **not by fixing the problem size**"*

- Corolario: aumentar el paralelismo puede permitir la modificación del problema original para ejecutar más trabajo
- Si el problema crece, caben dos alternativas:
Parte serial disminuye $\Rightarrow$ Speedup aumenta
Paralelismo aumenta $\Rightarrow$ Speedup aumenta





Paralelización de tareas | Modelo Work-Span

- **Características**

- Modelo más cercano a la realidad para estimar optimizaciones que el usado por Amdahl
- Provee una *cota inferior* y una *cota superior* para el *Speedup*

- **Hipótesis**

- *Paralelismo Imperfecto*: No todo el trabajo paralelizable se puede ejecutar al mismo tiempo
- *Greedy scheduling*: proceso disponible => tarea ejecutada
- Tiempo de acceso a memoria despreciable
- Tiempo de comunicación entre procesos despreciable
- Posibilidad de analizar la operación/algoritmo en caja blanca



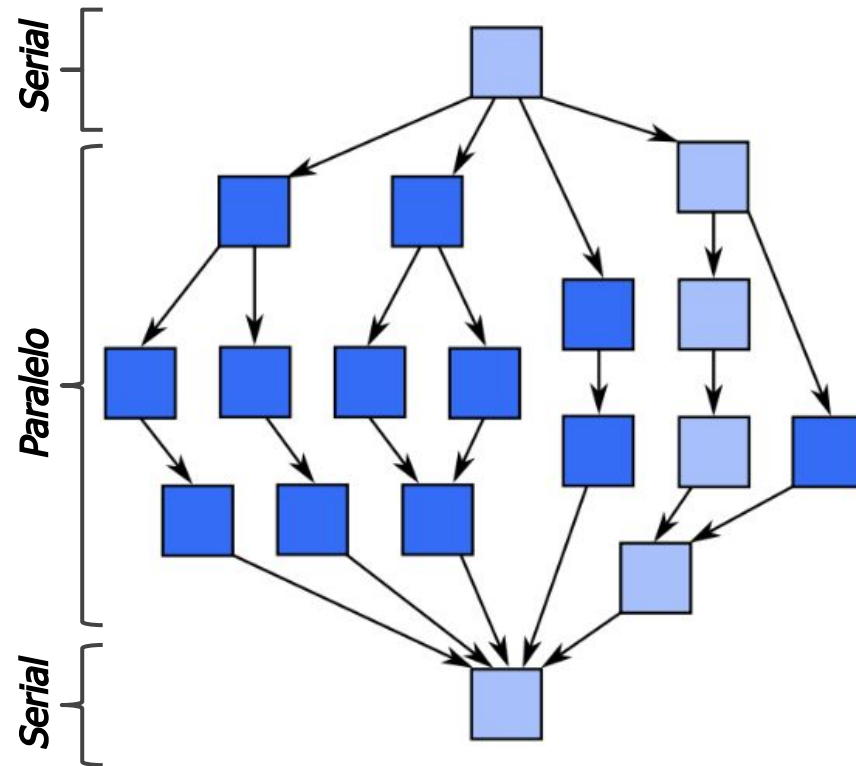
Paralelización de tareas | Modelo Work-Span

- Definiciones
 - $T_1(\text{work})$: Tiempo en ejecutar "*operación/algorithm*" con 1 sólo proceso
 - $T_{\text{inf}}(\text{span})$: Tiempo en ejecutar el **camino crítico** de la "*operación/algorithm*"

Cota	Speedup	Consideraciones
Superior	$\min(P, T_1/T_{\text{inf}})$	Se obtiene P en escenarios de Speedup lineal.
Inferior	$(T_1 - T_{\text{inf}})/P + T_{\text{inf}}$	El trabajo se puede dividir en perfecta e imperfectamente paralelizable



Paralelización de tareas | Modelo Work-Span

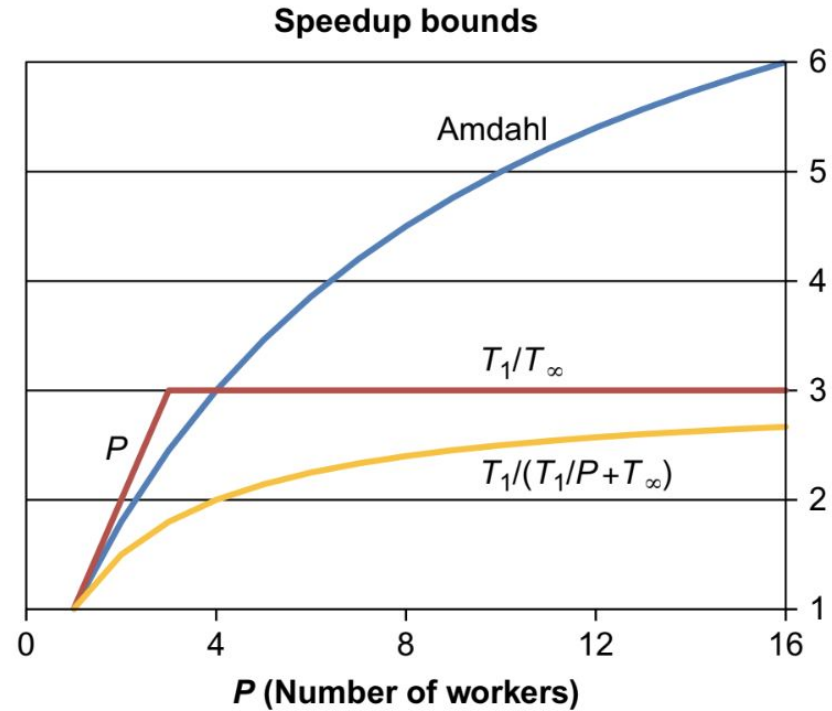
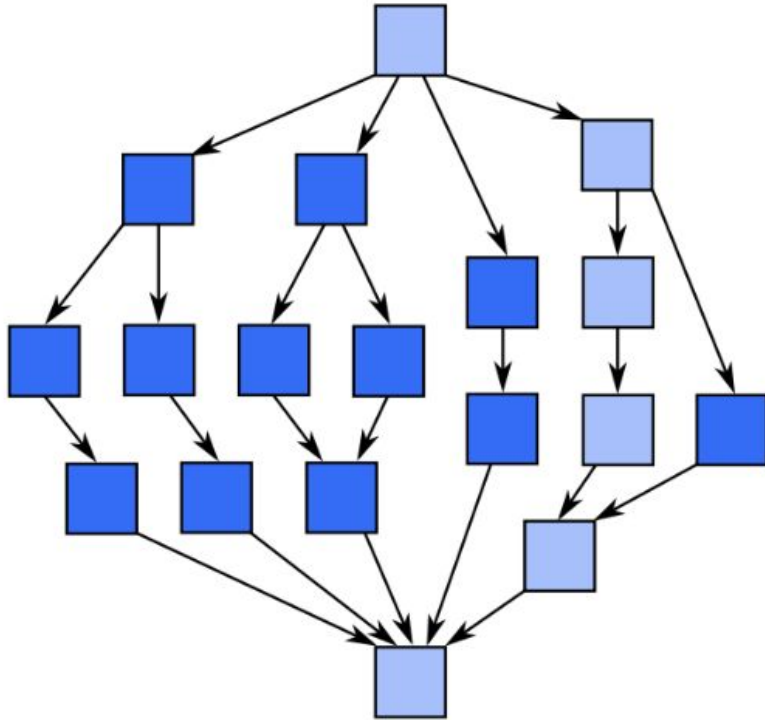


Ejemplo Práctico

- **Work-Span**
 - $T_1 = 18$ (work)
 - $T_{inf} = 6$ (span)
 - $S_p = T_1 / T_{inf} = \mathbf{3}$
- **Amdahl**
 - $f = 2/18 = 1/9$
 - $S_{inf} = 1/f = \mathbf{9}$



Paralelización de tareas | Modelo Work-Span





Estrategias de paralelización

- **Descomposición Funcional**

$\text{foo}(\text{data}) = f(\text{data}) + g(\text{data}) + h(\text{data})$ //1 proceso máximo

VS

$\text{foo}(\text{data}) = \text{go } f(\text{data}) + \text{go } g(\text{data}) + \text{go } h(\text{data})$ //3 procesos máx.

- **Particionamiento de Datos**

$\text{foo}(\text{data}) = f(\text{data})$ //1 proceso máx.

VS

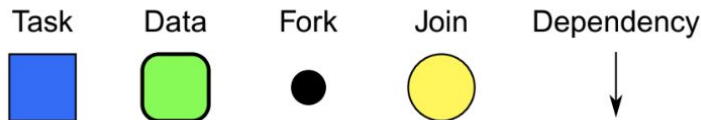
$\text{foo}(\text{data}) = \text{go } f(\text{data}[0:N/P-1]) \& \dots \& \text{go } f(\text{data}[(P-1)*N/P:N-1])$
//P procesos máx. (sólo si $f(x)$ es particionable)



Patrones de Procesamiento | Patrones de Procesamiento

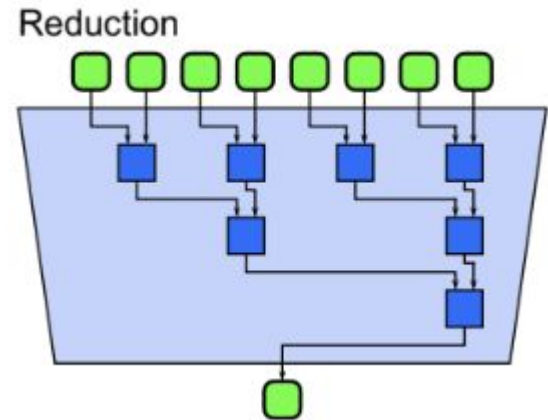
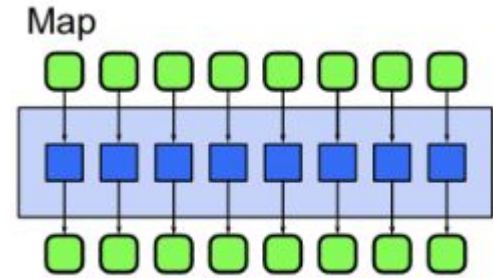
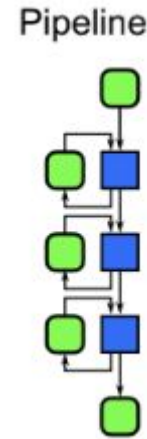
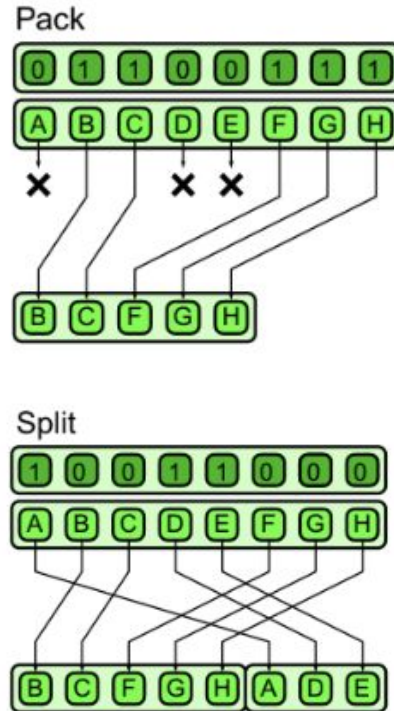
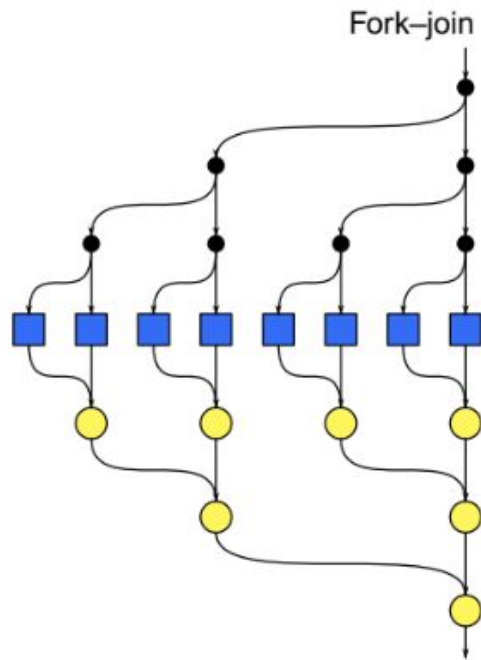
- Basados en algoritmos
 - No tan abstractos como *patrones de diseño*
 - No incluyen detalles de implementación
 - Agnósticos a lenguajes de programación
- Patrones deben poder incluir otros patrones (*nesting*)
- Herramientas básicas de trabajo también en *multi-computing*

Notación:





Paralelización de tareas | Patrones de Procesamiento



Agenda

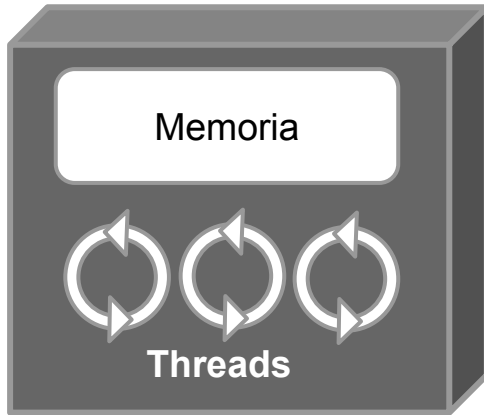


- Paralelización de Tareas
- **Multi-Processors & Multi-Computing**
- Nombres y Direccionamiento

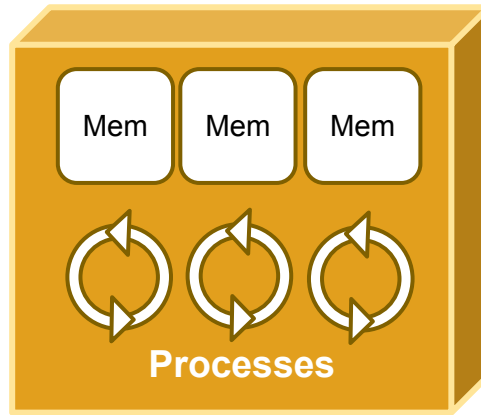


Modelos de Multiprogramación

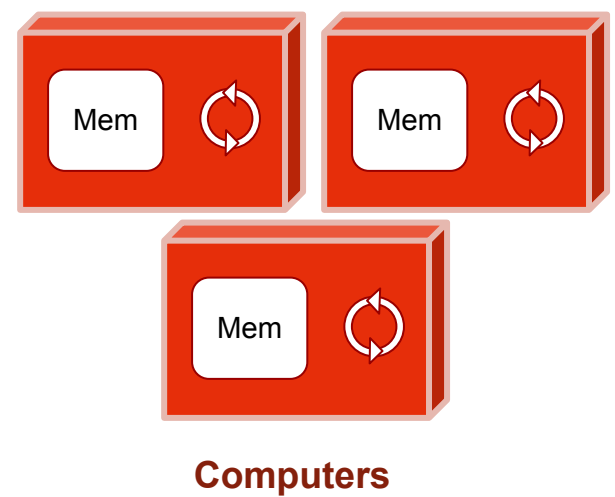
Multi-threading



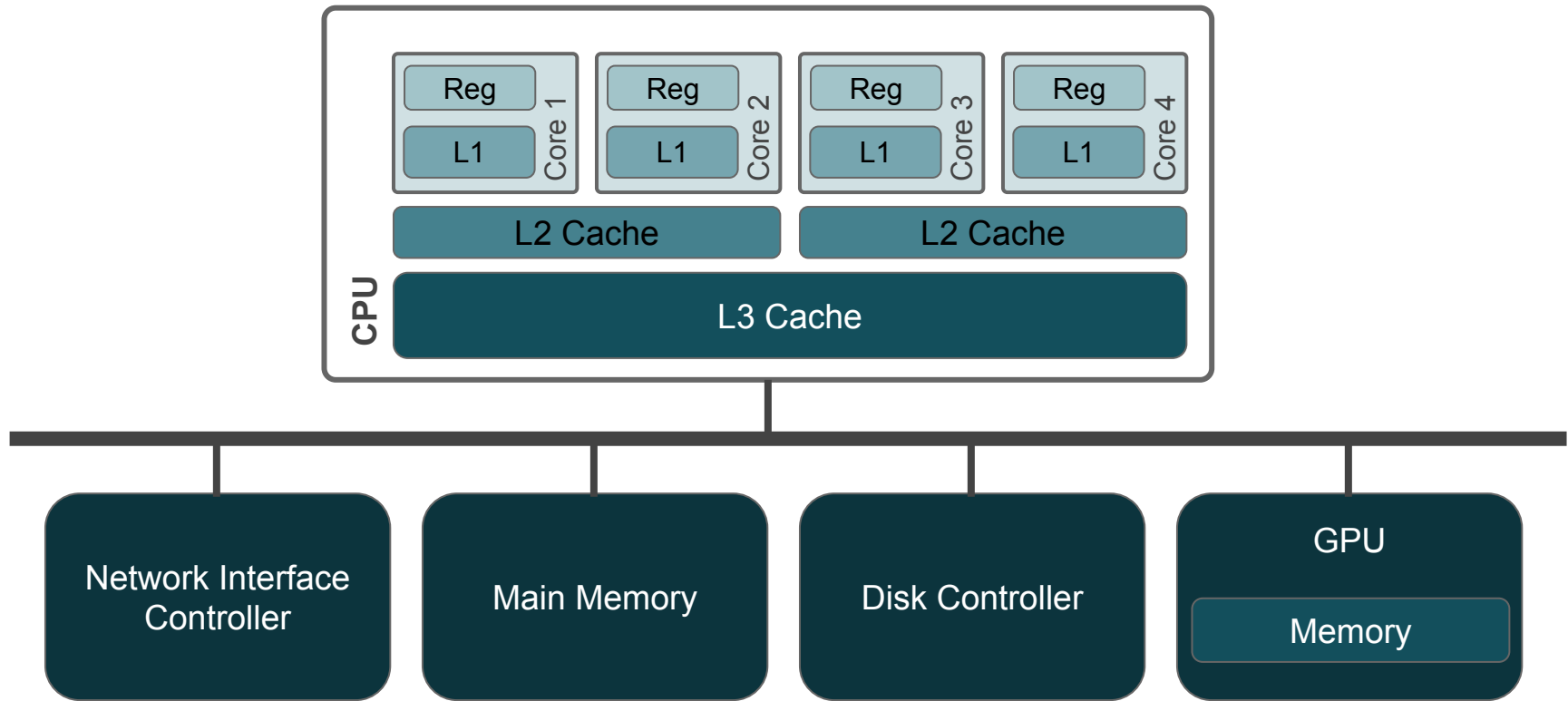
Multi-processing



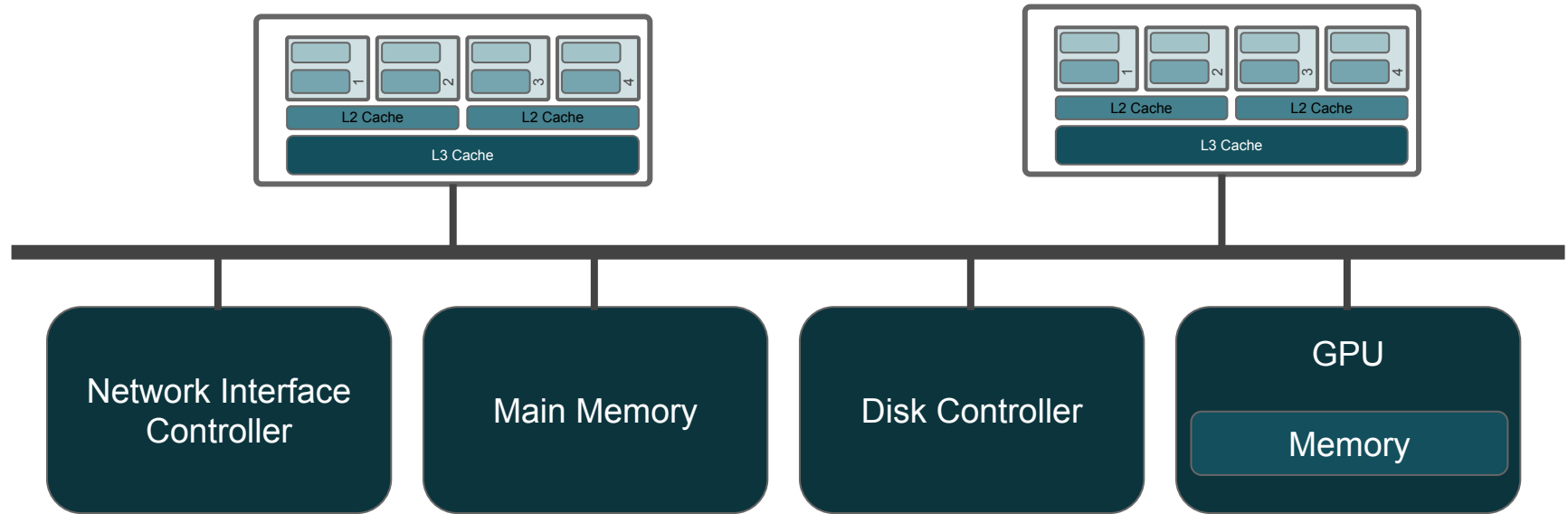
Multi-computing



Modelo Físico | Una CPU



Modelo Físico | Varias CPUs

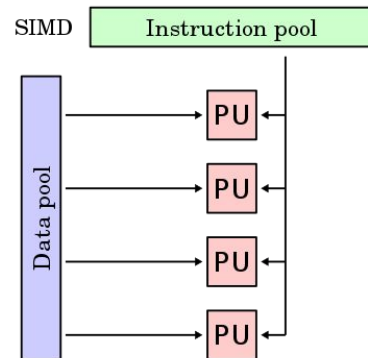
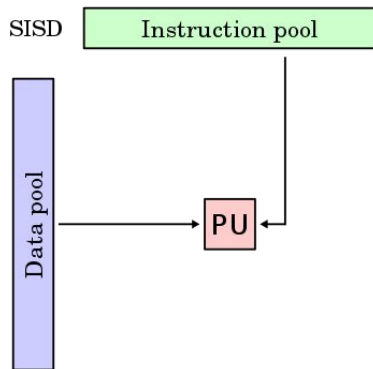


Muchos Procesadores | Taxonomía de Flynn



Clasificación de sistemas de acuerdo a cardinalidad de flujos de instrucciones (procesadores) y flujos de datos (memoria).

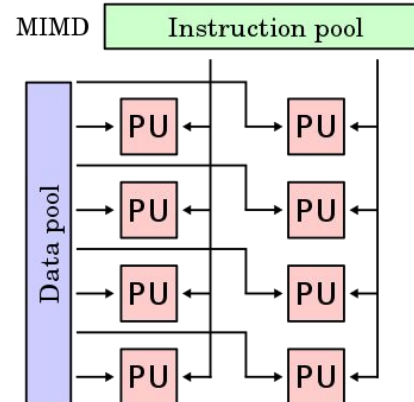
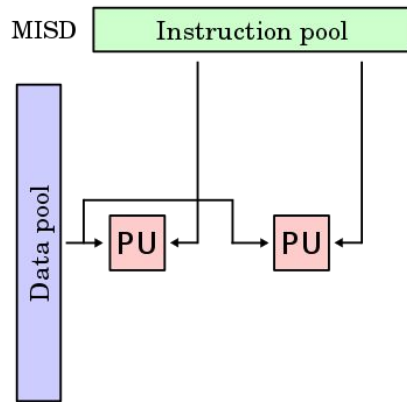
- **SISD** (*Single Instruction Single Data*): modelo estándar de un procesador sin paralelismo
- **SIMD**: *array processors*. Ej: GPU





Muchos Procesadores | Taxonomía de Flynn

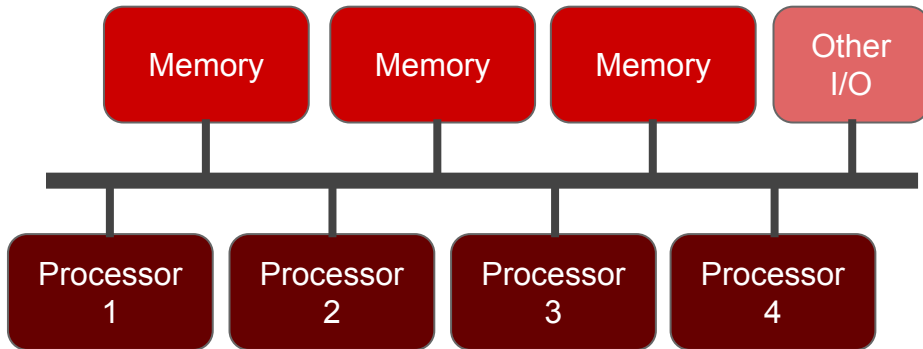
- **MISD:** no son usuales (*redundant computation*, data-pipelines)
- **MIMD:** se divide en dos modelos:
 - *Multiprocessors* (?): con memoria y/o *clock* compartidos
 - *Multicomputers*: sin memoria ni *clock* compartidos



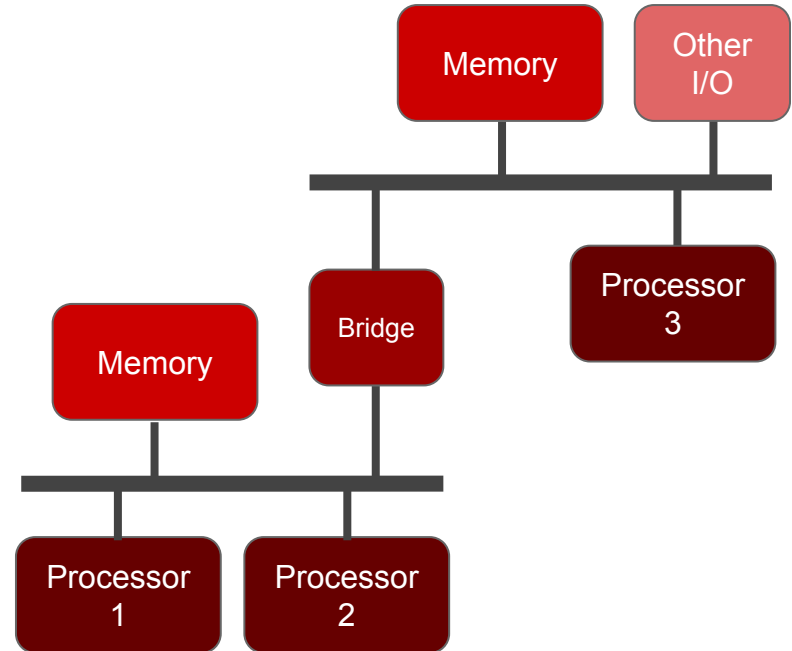
MIMD | *Multiprocessors* (Memoria Compartida)



Symmetric Multiprocessing

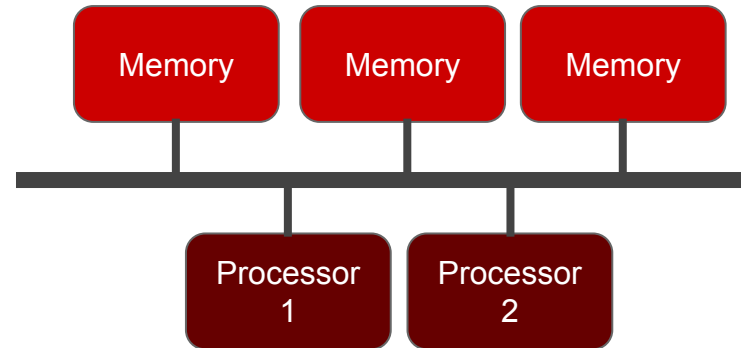


Asymmetric Multiprocessing



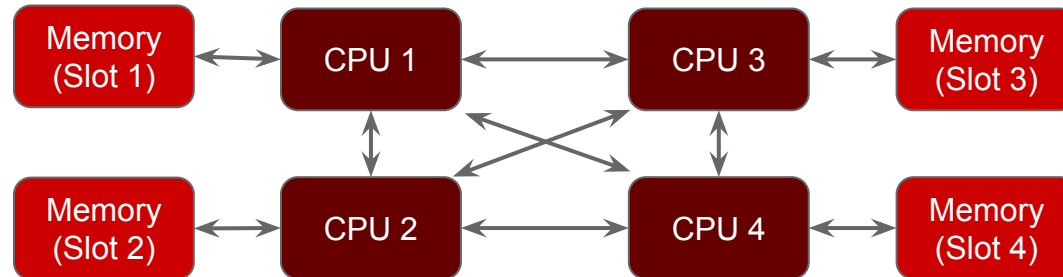


- Uniform Memory Access (**UMA == non-NUMA**)
 - Tiempo de acceso a la memoria es idéntico para todos los procesadores.
 - Ancho de banda compartido por todos.
 - *Performance* balanceada para aplicaciones de uso general.





- Non Uniform Memory Access (**NUMA**):
 - Cada CPU controla un bloque de memoria local como su 'home agent'.
 - Mayor ancho de banda si se respeta el acceso a memoria local.
 - Ideado en SGI, presente en Linux kernel y MS Servers.
 - Luego de años en desuso, nuevamente se ofrece en Cloud ([ver Ej.](#))





MIMD | *Multicomputers* (sin memoria compartida)

- Cada computadora tiene su propia memoria local
- Cada computadora puede fallar de forma independiente
- No poseen un reloj central de ejecución de instrucciones
- Requieren comunicación entre computadoras:
 - Networking: LAN, MAN, WAN



Recursos Compartidos

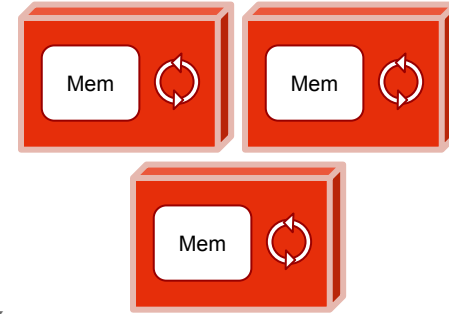
- Ninguno

Sincronización

- Mensajes Ad-Hoc entre Computadoras
=> necesidad de implementar mecanismos de sincronización.

Características clave

- Comunicación de red => problemas por limitaciones de ancho de banda, latencia y pérdida de mensajes.
- Comunicación entre procesos: compleja y central al diseño del sistema.
- Alta escalabilidad y tolerantes a fallos.



Agenda



- Paralelización de Tareas
- Multi-Processors & Multi-Computing
- **Nombres y Direccionamiento**



Nombres y Direccionamiento | Introducción

- **Nombres**
 - Permiten identificar *unívocamente a una entidad* dentro de un sistema
 - Deben describir a la entidad
 - Abstraen al recurso de las propiedades que atan al mismo con el sistema (lugar geográfico, direcciones de red)
- **Direccionamiento (Addressing)**
 - Mapeo entre un nombre y una dirección
 - Dirección de una entidad puede cambiar, **nombre no** (*)
 - Dirección puede ser reutilizada



Nombres y Direccionamiento | Ejemplos

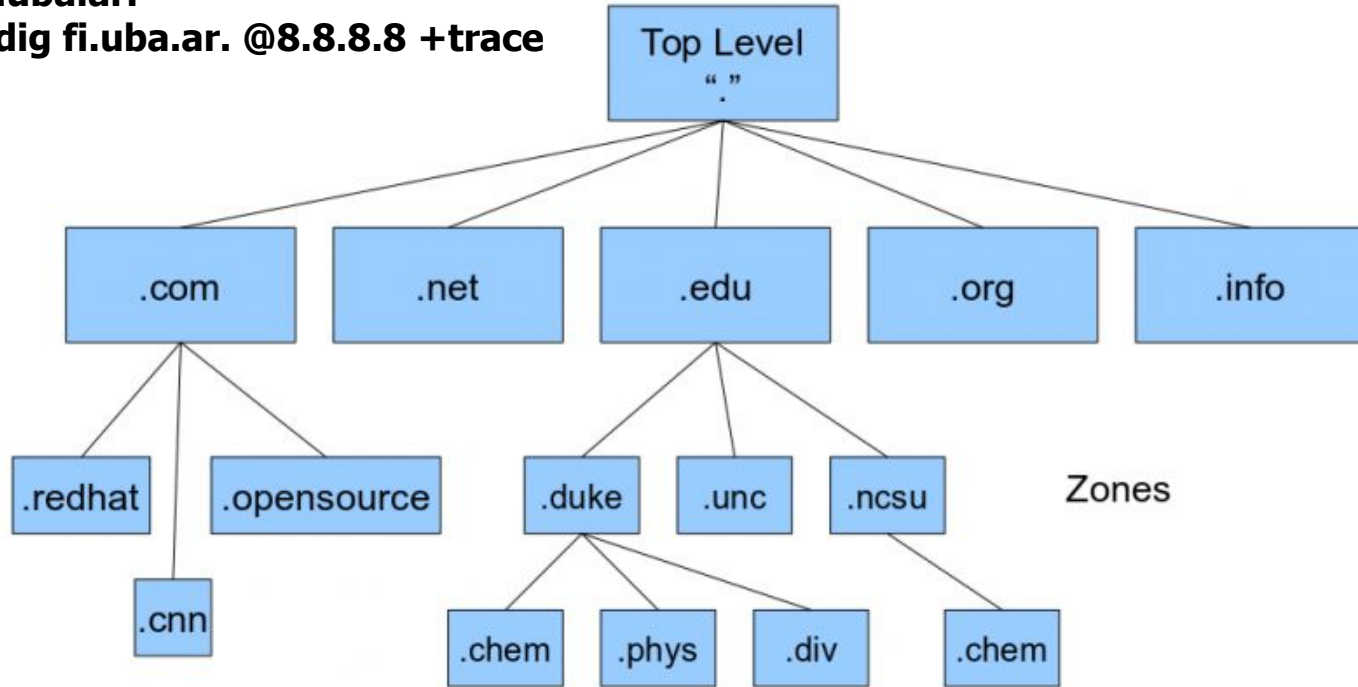
- **Domain Name (name) -> IP Address (address)**
 - Mapeo de un servicio/nodo/otra entidad a una dirección IP
 - Traducción a través de protocolo DNS
- **IP Address (name) -> Ethernet Address (address)**
 - IP address identifica a un nodo en un red (sea local o no)
 - Ethernet address identifica a NIC (network interface card) de un nodo en una red local
 - Resolución se realiza a través de protocolo ARP en IPv4 o [ND](#) (Neighbor Discovery) en IPv6
- **Service (name) -> Instances (address)**
 - Mapeo del nombre de un servicio a alguna instancia
 - Resolución a través de Service Discovery
 - Diferentes implementaciones existentes: Zookeeper, Istio, Linkerd



Nombres y Direcccionamiento | Ejemplo: DNS

Ejemplo: fi.uba.ar.

Comando: dig fi.uba.ar. @8.8.8.8 +trace





Nombres y Direcccionamiento | Ejemplo: Service Discovery

